مـدن الـمـهـن و الـكـفـاءات
Cités des métiers et des compétences

OFPPT
La voie de l'avenir

مكتب التكوين المهني و إنعاش الشغل
Office de la Formation Professionnelle
et de la Promotion du Travail

Module: Développer en back-end

Base de données
Query Builder

Filière: Développement digital – option web full stack

Développer en back-end

C. Approfondir la programmation Laravel

2. Interagir avec la base de données

Générateur de requêtes (Query Builder)

Introduction

Le générateur de requêtes de base de données de Laravel fournit une interface pratique et fluide pour créer et exécuter des requêtes de base de données.

Il peut être utilisé pour effectuer la plupart des opérations de base de données dans votre application et fonctionne parfaitement avec tous les systèmes de base de données pris en charge par Laravel.

Le générateur de requêtes Laravel utilise la liaison de paramètres **PDO** pour protéger votre application contre les attaques par injection SQL.

Il n'est pas nécessaire de nettoyer ou d'assainir les chaînes transmises au générateur de requêtes en tant que liaisons de requête.

C. Approfondir la programmation Laravel

2. Interagir avec la base de données

Générateur de requêtes (Query Builder)

La configuration de la base de données

Indiquer le nom de la base de données cmc dans le fichier de configuration **.env** :

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=cmc
DB_USERNAME=root
DB_PASSWORD=
```

Exécution de migrations

lancer les migrations on utilise la commande migrate :

```
php artisan migrate
```

générer la migration suivante :

```
php artisan make:migration
create_stagiaires_table
```

C. Approfondir la programmation Laravel

2. Interagir avec la base de données

Générateur de requêtes (Query Builder)

Les fonctions (**up et down**) de la migration précédente :

```
public function up()
    {
        Schema::create('stagiaires', function (Blueprint $table) {
            $table->id();
            $table->string('nom', 50);
            $table->string('prenom', 50);
            $table->integer('age');
            $table->timestamps();
        });
    }
```

```
public function down()
    {
        Schema::dropIfExists('stagiaires');
    }
```

Développer en back-end

C. Approfondir la programmation Laravel

2. Interagir avec la base de données

Générateur de requêtes (Query Builder)

Insert, Update et Delete

Créer un contrôleur **StagiaireController** et ajouter la méthode suivante :

```
use Illuminate\Support\Facades\DB;
```

La méthode `insert()` permet d'insérer une ligne dans une table.

```
public function insert(Request $request){

        DB::table('stagiaires')->insert([
            'nom'=>$request->nom,
            'prenom'=>$request->prenom,
            'age'=>$request->age
        ]);
        return response("l'ajout est effectué avec succès ... ");
    }
```

C. Approfondir la programmation Laravel

2. Interagir avec la base de données

Générateur de requêtes (Query Builder)

Insert, Update et Delete

La méthode **`insertGetId()`** fait le même chose que **`insert()`** mais en plus vous permet de récupérer l'**id** créé pour cette enregistrement. Bien évidement **`insertGetId()`** ne marche qu'avec des tables ayant un **id** en **auto incrémentation**.

```
public function insert(Request $request){

    $id=DB::table('stagiaires')->insertGetId([
            'nom'=>$request->nom,
            'prenom'=>$request->prenom,
            'age'=>$request->age
        ]);
      return response("l'ajout est effectué avec succès ... ".$id);
    }
```

C. Approfondir la programmation Laravel

2. Interagir avec la base de données

Générateur de requêtes (Query Builder)

Insert, Update et Delete

La méthode **`update()`** permet de mettre à jour des données.

```
public function update(Request $request){

      DB::table('stagiaires')->where('id',$request->id)->update([
            'nom'=>$request->nom,
            'prenom'=>$request->prenom,
            'age'=>$request->age
       ]);
  return response("la mise à jour est effectué avec succès ... ");
    }
```

C. Approfondir la programmation Laravel

2. Interagir avec la base de données

Générateur de requêtes (Query Builder)

Insert, Update et Delete

La méthode **`delete()`** permet de supprimer un enregistrement

```
public function delete(Request $request){

  DB::table('stagiaires')->where('id',$request->id)->delete();
  return response("la supression est effectué avec succès ... ");
   }
```

C. Approfondir la programmation Laravel

2. Interagir avec la base de données

Générateur de requêtes (Query Builder)

Sélectionner des résultats

le Query Builder est accessible via la façade **DB** et permet de sélectionner une table puis utiliser des méthodes pour recèperez les données.

```
// Rappelle de la structure de départ
 DB::table('stagiaires') // je prépare une requête pour la table 'stagiaires'
```

La méthode `get()` permet de récupérer un groupe de résultat.

```
public function index(){
        $sta=DB::table('stagiaires')->get();
        return $sta;
   }
```

C. Approfondir la programmation Laravel

2. Interagir avec la base de données

Générateur de requêtes (Query Builder)

Sélectionner des résultats

La méthode **where()** permet de poser une condition à la sélection de résultat :

```
$sta=DB::table('stagiaires')-> where('age','>',20)->get();
```

La méthode **first()** a un fonctionnement similaire à get() mais ne récupère que le premier résultat correspondant à la requête.

```
$sta=DB::table('stagiaires')->first();
```

La méthode **find()** permet de sélectionner un résultat d'après un **id** que l'on passe en paramètre :

```
$sta=DB::table('stagiaires')->find(id);
```

C. Approfondir la programmation Laravel

2. Interagir avec la base de données

Générateur de requêtes (Query Builder)

Sélectionner des résultats

La méthode **orderBy()** Permet d'organiser les résultats en les rangeant dans un order ascendant (ASC) ou descendant (DESC) d'après une colonne de la table :

```
$sta=DB::table('stagiaires')->orderBy('nom', 'DESC')->get();
```

La méthode **groupBy** permet de grouper les résultats par valeur(s) d'une ou plusieurs colonnes.

```
$sta=DB::table('stagiaires')->groupBy('age')->get();
```

C. Approfondir la programmation Laravel

2. Interagir avec la base de données

Générateur de requêtes (Query Builder)

Sélectionner des résultats

Les méthodes **`latest()`** et **`oldest()`** rangent les données par date en prenant comme référence par défaut la colonne '**created_at**'.

Si vous voulez appliquer ces méthodes à une autre colonne vous n'avez qu'à passer en paramètre le nom de celle-ci.

Ici je n'ai pas mis de timestamps mais nous pouvons par exemple appliquer cette méthode avec la colonne '**id**'.

```
$sta=DB::table('stagiaires')->latest('id')->first();
```

La méthode **`inRandomOrder()`** vous permet tout simplement de sortir vos résultats dans un ordre aléatoire.

C. Approfondir la programmation Laravel

2. Interagir avec la base de données

Générateur de requêtes (Query Builder)

Opérations numérique

Vous pouvez également appliquer des opérations numériques existantes dans les commandes SQL :

```php
$max_age = DB::table('stagiaires')->max('age');
$min_age = DB::table('stagiaires')->min('age');
$average_age = DB::table('stagiaires')->avg('age');
$sum_age = DB::table('stagiaires')->sum('age');
```

À savoir !

*Nous venons de voir un ensemble de méthode appliquées à la façade **DB** mais sachez que ces méthodes sont également disponible via le **model***